

Conversational Robot

System Architecture Plan v2

Raspberry Pi 5 (8GB) · Gemma 4 E4B · OpenClaw · llama.cpp · Piper TTS

01	Hardware & Software Stack
02	System Architecture — Layers
03	OpenClaw Tool Schema
04	Latency Budget
05	Boot & Startup Sequence
06	Repository Structure
07	Constraints & Mitigations

01

Hardware & Software Stack

COMPUTE

Raspberry Pi 5 — 8GB RAM

Quad-core Cortex-A76 @ 2.4GHz · Active cooler installed · 65GB microSD storage

LLM BRAIN

Gemma 4 E4B (Q4_K_M quantisation)

~4GB RAM footprint · Vision + Audio + Native tool calls · 128K context window

INFERENCE ENGINE

llama.cpp (ARM NEON optimised)

10–20% faster than Ollama on Pi 5 · Full thread & context window control

AGENT FRAMEWORK

OpenClaw daemon

Constant stimulus polling · Tool call dispatch · Persistent event loop

WAKE WORD

openWakeWord

Always-on background listener · Triggers agent loop · Near-zero CPU in idle

TTS

Piper TTS (streaming)

Sentence-level streaming synthesis · Fully on-device · USB speakers output

02

System Architecture — Layers

LAYER 1 — PERCEPTION		ALWAYS ON
Wake Word Detection openWakeWord runs on a dedicated background thread. Listens for 'Hey Robot'. Near-zero CPU in idle state. On detection: fires event onto OpenClaw event bus. Config: hey_robot.onnx · threshold 0.7 · chunk 1280	Audio & Vision Capture USB microphone via PyAudio, VAD-terminated recording. Pi Camera Module 3 snapshot taken in parallel with audio. Both passed inline to Gemma 4 — no separate transcription step or snapshot pipeline required. Audio: 16kHz mono PCM · Camera: 640×480 JPEG	
LAYER 2 — AGENT ORCHESTRATION		OPENCLAW
Event Loop systemd service, starts on boot. Polls all stimuli sources every 50ms. On event: packages audio + image into multimodal prompt, dispatches to llama.cpp HTTP server, receives streamed tool calls or text response.	Prompt Construction Assembles multimodal prompt per invocation. Injects: system prompt, tool schema, persistent memory, last 6 turns of history, audio, camera frame. Context budget capped at 4096 tokens to maintain fast inference on the Pi 5.	
LAYER 3 — INFERENCE		LLAMA.CPP
llama.cpp HTTP Server Runs on localhost:8080. Model loaded once on boot, stays warm in RAM (~4GB) for the session. Streaming mode: tokens stream back as generated, allowing TTS to begin before inference finishes. Flags: -c 4096 -t 4 --n-predict 256 --no-mmap	Tool Call Parsing Gemma 4 natively outputs JSON tool calls. OpenClaw parses the streamed output in real time. If tool call detected: execute tool, inject result back as next user turn (max 3 hops per turn). If plain text: pipe directly to Piper TTS.	
LAYER 4 — OUTPUT		PIPER TTS
Piper TTS — Streaming Receives text from OpenClaw sentence-by-sentence. Synthesises audio chunks, pipes to aplay → speakers. First audio out within ~150ms of first sentence. Robot begins speaking before generation is complete.	Acknowledgment Tone 200ms tone plays immediately on wake word detection. Confirms the robot heard before LLM inference starts. Eliminates the dead-silence window during ~1s TTFT. Pre-rendered WAV file — playback is instant.	

03

OpenClaw Tool Schema

Gemma 4 E4B supports native function calling. The following tools are registered in tools/registry.json and injected into the system prompt on every turn.

TOOL	TRIGGER	PARAMETERS	DESCRIPTION
speak()	Any verbal response	text: str voice: str speed: float	Routes text to Piper TTS. Streaming: starts before LLM finishes generating.
describe_scene()	Visual / 'what do you see?'	focus: str (optional)	Fresh camera snapshot + vision prompt. Returns structured scene description.
remember()	Facts to retain across turns	key: str value: str	Writes to memory.json. Injected into system prompt on every invocation.
get_time()	Time / date queries	(none)	Returns Pi system clock datetime. Prevents Gemma hallucinating the date.
set_reminder()	"Remind me to..."	message: str delay_seconds: int	Schedules future speak() via OpenClaw scheduler. Robot speaks proactively.
gpio_signal()	Physical action commands	pin: int state: bool duration_ms: int?	Writes to GPIO pins. Connect LEDs, servos, or relays. Extensible layer.

04

Latency Budget

End-to-end timing from wake word detection to the first word spoken by the robot.

STAGE	ESTIMATED TIME	NOTES
Wake word detection → signal	~50 ms	openWakeWord internal detection latency
Acknowledgment tone playback	~200 ms	Pre-rendered WAV, instant playback
Audio recording (user speaks)	1 – 3 s	VAD-terminated, controlled by user
Camera snapshot	~80 ms	Runs in parallel during audio recording
Prompt construction + dispatch	~30 ms	OpenClaw assembly + local HTTP call
LLM first token (TTFT)	~800 ms – 1.2 s	Gemma 4 E4B cold prompt eval on Pi 5
First sentence generation	~1 – 2 s	~15–20 tokens @ 8–10 tok/sec
Piper TTS first audio out	~150 ms	Streaming, starts on first sentence
Total: wake → first spoken word	~3 – 5 s	Dominated by user speech + TTFT

05

Boot & Startup Sequence

The robot stack runs as a systemd service. On power-on, llama.cpp loads the model from microSD (~30s), then OpenClaw starts and the robot is ready.

```
# /etc/systemd/system/robot.service

[Unit]
Description=Conversational Robot Stack
After=network.target

[Service]
ExecStartPre=/home/pi/robot/scripts/start_llama_server.sh
ExecStart=/home/pi/robot/openclaw/main.py
Restart=always

# start_llama_server.sh

llama-server -m /home/pi/models/gemma-4-e4b-q4_k_m.gguf \
--host 127.0.0.1 --port 8080 -c 4096 -t 4 &

sleep 30 # wait for model to load from microSD
```

06

Repository Structure

```
robot/
├── models/
│   ├── gemma-4-e4b-q4_k_m.gguf # ~4 GB
│   └── openclaw/
│       ├── main.py # daemon entry point
│       ├── event_loop.py # polling + dispatch
│       ├── prompt_builder.py # multimodal prompt assembly
│       ├── tool_parser.py # streaming JSON tool call parser
│       └── tools/
│           ├── registry.json # Gemma 4 tool schema
│           ├── speak.py
│           ├── vision.py
│           ├── memory.py
│           ├── gpio.py
│           └── perception/
│               ├── wake_word.py # openWakeWord wrapper
│               ├── audio_capture.py # PyAudio + silero VAD
│               └── camera.py # Pi Camera Module 3
```

```
■■■ tts/
■ ■■■ piper_stream.py # streaming Piper wrapper
■■■ memory.json # persistent key-value store
■■■ robot_system_prompt.txt
■■■ scripts/
■■■ start_llama_server.sh
```

07

Constraints & Mitigations

01

SD Card Cold Boot (~30–45s)

The model loads from microSD once on startup and stays warm in RAM for the entire session. This is not a runtime issue — it only affects cold boot. For a class demo where you power on once and leave it running, this is fully acceptable.

02

Thermal Throttling Under Sustained Load

The active cooler mitigates this significantly. Monitor temperature with `vcgencmd measure_temp`. If the CPU exceeds 80°C, inference drops from ~10 tok/sec to ~5 tok/sec. For sustained demos, allow a cooldown break between long sessions.

03

8–10 tok/sec Is Sufficient for Speech

Piper TTS reads aloud at ~120 words/min (~2 tokens/sec audible output). The LLM generates tokens faster than the robot can physically speak them, so the pipeline never bottlenecks on inference speed in normal conversational use.

04

Fully Offline — No Network Dependency

Wake word detection, LLM inference, TTS synthesis, and all tool execution run entirely on-device. The robot is robust to demo environments with unreliable Wi-Fi. No API keys required. No data leaves the Pi.
